

LAB MANUAL

Topic: n8n Workflow Automation

Subject: Workflow Automation / DevOps Tools

Objective:

The objective of this lab manual is to help students understand the concept of Workflow Automation and get hands-on with n8n, a popular open-source workflow automation tool. After completing this lab, students will be able to explain the difference between traditional automation and workflow automation, describe what n8n is and why it is used, identify the key features and use cases of n8n, navigate the n8n interface, install n8n on Windows, Ubuntu, and using Docker, understand localhost and ports in the context of n8n, and start, stop, and restart an n8n instance.

Part 1: Introduction to Workflow Automation

1.1 What is Workflow Automation?

Workflow Automation is the process of designing a sequence of automated steps (a *workflow*) that connects different applications, services, and data sources so that a task, which would normally require repeated manual effort, runs on its own — triggered by an event, a schedule, or a webhook. A workflow typically consists of a **trigger** (what starts the workflow, e.g., a new form submission) and one or more **actions** (what happens next, e.g., send an email, add a row to a spreadsheet, post a Slack message).

1.2 Traditional Automation vs Workflow Automation

Traditional automation usually means writing a standalone script or a scheduled job (e.g., a cron job or a batch file) that automates **one specific task** in isolation. Workflow automation, on the other hand, connects **multiple applications and services together** visually, without necessarily writing code, allowing data to flow between them as part of one coordinated process.

Aspect (Traditional)	Description
Approach	Requires custom scripts/code for every task, written and maintained by a developer.
Scope	Usually automates a single task or system in isolation.
Integration	Connecting to another service (API, database, app) requires writing custom integration code.
Visibility	Logic is hidden inside code files; hard for non-developers to understand or modify.
Maintenance	Any change requires editing and redeploying code.

Example	A Python cron script that reads a CSV file every night and emails a summary report.
Aspect (Workflow Automation)	Description
Approach	Uses a visual, node-based canvas to design workflows; little or no code required.
Scope	Connects and coordinates many different apps/services as one end-to-end process.
Integration	Ships with hundreds of pre-built connectors ("nodes") for common apps and APIs.
Visibility	The whole logic is visible as a flow diagram, understandable by non-developers too.
Maintenance	Workflows can be edited by dragging, dropping, and reconfiguring nodes.
Example	An n8n workflow that watches a Google Form, adds the response to a spreadsheet, sends a Slack notification, and emails a confirmation — all in one visual flow.

1.3 What is n8n?

n8n (pronounced "*n-eight-n*", short for "nodemation") is a free and open-source workflow automation tool. It provides a visual, node-based canvas where each **node** represents a step — a trigger, an action, or a piece of logic. Nodes are connected together to form a **workflow**, which n8n executes automatically whenever it is triggered. Unlike many closed SaaS automation tools, n8n can be **self-hosted** (run on your own machine, server, or Docker container), giving full control over data and infrastructure, while also supporting custom JavaScript/Python code inside a workflow when a built-in node isn't enough.

1.4 Why use n8n for Automation?

- **Open-source and self-hostable:** Can be run for free on your own infrastructure, keeping data private and avoiding per-task pricing common in closed SaaS tools.
- **Visual workflow builder:** Workflows are built by dragging and connecting nodes on a canvas, making automation accessible to non-programmers.
- **Extensible with code:** A "Code" node allows JavaScript or Python snippets inside a workflow for custom logic that pre-built nodes don't cover.
- **Large node/integration library:** Built-in nodes for common apps (Slack, Gmail, Google Sheets, databases, HTTP APIs, etc.) mean most integrations don't need to be written from scratch.

- **Fair-code license:** The source code is visible and self-hostable, unlike fully closed commercial tools, while still offering a paid cloud/enterprise option for teams that want it.
- **Flexible triggers:** Workflows can start from a schedule, a webhook, an app event, a form submission, a manual click, or a chat message.

1.5 Key Features and Use Cases of n8n

Feature	Description
Visual Workflow Editor	Drag-and-drop canvas for building automations by connecting nodes.
400+ Integrations	Pre-built nodes for popular apps, databases, and APIs (Slack, Gmail, Sheets, Postgres, etc.).
Webhook & Trigger Nodes	Start workflows from HTTP requests, schedules (cron), or app-specific events.
Code Node	Write custom JavaScript or Python directly inside a workflow for advanced logic.
Error Handling	Dedicated error-workflow and retry options to gracefully handle failures.
Credentials Manager	Securely stores and reuses API keys/OAuth tokens across workflows.
Self-Hosting Options	Runs via npm, Docker, or Docker Compose on Windows, macOS, Linux, or a server.

Typical use cases include:

- Syncing new leads from a web form into a CRM and notifying the sales team on Slack.
- Automatically backing up database records to Google Sheets or a cloud storage bucket every night.
- Building simple internal chatbots or AI agents that call external APIs.
- Sending automated email/SMS reminders based on calendar or database events.
- Aggregating data from multiple APIs into a single report or dashboard.

1.6 Understanding the n8n Interface

Once n8n is running and opened in the browser, the main interface (the **Editor UI**) consists of the following areas:

Interface Element	Purpose
Canvas	The large central area where nodes are placed and connected to build a workflow.
Nodes Panel	Opened with the "+" button; lets you search and add trigger/action nodes to the canvas.

Node Panel (Editor)	Opens when a node is double-clicked; used to configure its parameters and credentials.
Trigger Node	The starting node of a workflow (e.g., Manual Trigger, Webhook, Schedule Trigger) - shown with a bolt/lightning icon.
Connections	The lines drawn between nodes showing the direction data flows through the workflow.
Execution Log / Output Panel	Shows the data (JSON) produced by each node after a workflow run, useful for debugging.
Top Bar	Contains Save, Active/Inactive toggle, Share, and workflow name/rename options.
Left Sidebar	Navigation between Workflows, Credentials, Executions, and Settings.

Part 2: n8n Installation & Setup

Required Software:

- Node.js (version 20.19 to 24.x) and npm — required for the npm-based installation.
- Docker Desktop (Windows/Mac) or Docker Engine (Ubuntu) — required for the Docker-based installation.
- A modern web browser (Chrome, Edge, Firefox) to access the n8n Editor UI.
- A terminal / command prompt (PowerShell, CMD, or a Linux shell).

2.1 Complete Setup Guide: n8n on Windows

n8n can be installed on Windows either via **npm** (after installing Node.js) or via **Docker Desktop**. The npm method below is the quickest way to get started locally.

Step 1: Install Node.js

Download and install the Node.js LTS installer (version 20 or 22 LTS) from nodejs.org, and run the installer with default settings. This also installs npm automatically.

Step 2: Verify the Installation

Open PowerShell or Command Prompt and confirm Node.js and npm were installed correctly:

```
node -v  
npm -v
```

Step 3: Install n8n Globally

Install n8n as a global npm package so the `n8n` command becomes available anywhere in the terminal:

```
npm install n8n -g
```

Step 4: Start n8n

Launch n8n from PowerShell/CMD:

```
n8n start
```

The terminal will print a message once n8n is ready, referencing the local editor URL.

Step 5: Open the Editor UI

Open a browser and navigate to the address below to access the n8n Editor UI and start building workflows:

```
http://localhost:5678
```

2.2 Complete Setup Guide: n8n on Ubuntu

On Ubuntu, the recommended approach is to install a supported Node.js version via NodeSource (or nvm), then install n8n with npm.

Step 1: Update the System

Refresh the package index before installing anything new:

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Node.js (via NodeSource)

Add the NodeSource repository for a supported Node.js LTS release and install it:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -  
sudo apt install -y nodejs
```

Step 3: Verify Node.js and npm

Confirm both were installed correctly:

```
node -v  
npm -v
```

Step 4: Install n8n Globally

Install n8n as a global npm package:

```
sudo npm install n8n -g
```

Step 5: Start n8n

Run n8n from the terminal:

```
n8n start
```

On first run, n8n creates a configuration folder at ~/.n8n to store credentials and workflow data.

Step 6: Open the Editor UI

Open a browser (or forward the port if working on a remote server) and go to:

```
http://localhost:5678
```

2.3 Running n8n with Docker

Docker is the recommended way to self-host n8n because it bundles the correct Node.js version and all dependencies inside a container, avoiding version-compatibility issues on both Windows and Ubuntu.

Step 1: Install Docker

Install Docker Desktop (Windows/Mac) or Docker Engine (Ubuntu), then confirm it is working:

```
docker --version
```

Step 2: Create a Volume for Persistent Data

Create a named Docker volume so workflows and credentials survive container restarts:

```
docker volume create n8n_data
```

Step 3: Run the n8n Container

Pull the official n8n image and start the container, mapping port 5678 and mounting the volume created above:

```
docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

Step 4: Access n8n

Once the container has started, open a browser and go to:

```
http://localhost:5678
```

Optional: Running n8n with Docker Compose

For a setup that is easy to start, stop, and restart repeatedly, create a `docker-compose.yml` file:

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    restart: always
    ports:
      - "5678:5678"
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
```

Then start it in the background with:

```
docker compose up -d
```

2.4 Understanding Localhost and Ports

Localhost (127.0.0.1) refers to the current machine itself — when n8n's URL is opened as `http://localhost:5678`, the browser is asking the same computer to serve that page, not a remote server. A **port** is a numbered communication endpoint on a machine; a single computer can run many applications at once, each listening on its own port, so the operating system knows which application should receive incoming traffic. n8n listens on **port 5678** by default.

- **`http://localhost:5678`** — the default local address for the n8n Editor UI.
- If port 5678 is already used by another application, n8n will fail to start until the conflict is resolved or a different port is configured.
- The default port can be changed using the **`N8N_PORT`** environment variable.

- When running n8n inside Docker, the **-p HOST_PORT:CONTAINER_PORT** flag maps a port on the host machine to port 5678 inside the container (e.g., -p 8080:5678 makes n8n reachable at localhost:8080).

Example: changing the default port on Windows/Ubuntu (npm install):

```
# Linux / macOS (bash)
export N8N_PORT=8080
n8n start

# Windows (PowerShell)
$env:N8N_PORT=8080
n8n start
```

2.5 Starting, Stopping and Restarting n8n

Method / Action	Command
npm installation - Start	Run <code>`n8n start`</code> (or simply <code>`n8n`</code>) in the terminal.
npm installation - Stop	Press <code>`Ctrl + C`</code> in the terminal window where n8n is running.
npm installation - Restart	Stop with <code>`Ctrl + C`</code> , then run <code>`n8n start`</code> again.
Docker (docker run) - Start	Run the <code>`docker run ...`</code> command shown in Part 2.3.
Docker (docker run) - Stop	Run <code>`docker stop n8n`</code> in another terminal (or <code>`Ctrl + C`</code> if run in the foreground).
Docker (docker run) - Restart	Run <code>`docker restart n8n`</code> .
Docker Compose - Start	Run <code>`docker compose up -d`</code> from the project folder.
Docker Compose - Stop	Run <code>`docker compose down`</code> (container is removed; data stays in the named volume).
Docker Compose - Restart	Run <code>`docker compose restart`</code> .

Note: For n8n to keep running after closing the terminal or logging out (e.g., on a server), use a process manager such as **pm2** (for npm installs) or Docker's ``restart: always`` policy (for Docker/Compose installs), covered in the Solved Tasks below.

Solved Task 1: Verify Node.js and npm Before Installing n8n

Problem:

Check whether Node.js and npm are installed and confirm their versions meet n8n's requirement (Node.js 20.19 - 24.x).

Solution:

```
node -v  
npm -v
```

Output:

```
v22.14.0  
10.9.2
```

Explanation: Both commands print the installed version numbers. If either command is not recognized, Node.js/npm needs to be (re)installed before n8n can be installed.

Solved Task 2: Install n8n Globally with npm and Start It

Problem:

Install n8n globally using npm, then start it and confirm it is serving the Editor UI.

Solution:

```
npm install n8n -g  
n8n start
```

Output:

```
Initializing n8n process  
n8n ready on 0.0.0.0, port 5678  
Editor is now accessible via:  
http://localhost:5678
```

Explanation: `npm install n8n -g` downloads and installs n8n as a global command-line tool; `n8n start` boots the server and prints the local URL once it is ready to accept connections.

Solved Task 3: Try n8n Instantly with npx (No Installation)

Problem:

Run n8n without permanently installing it, useful for a quick test.

Solution:

```
npx n8n
```

Output:

```
Downloading n8n packages... done
n8n ready on 0.0.0.0, port 5678
Editor is now accessible via:
http://localhost:5678
```

Explanation: `npx` downloads n8n temporarily and runs it immediately, without adding a permanent global package — ideal for a first look before committing to a full install.

Solved Task 4: Run n8n Using Docker with Persistent Data

Problem:

Start n8n inside a Docker container so that workflow data survives container restarts.

Solution:

```
docker volume create n8n_data

docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

Output:

```
Unable to find image 'docker.n8n.io/n8nio/n8n:latest' locally
latest: Pulling from n8nio/n8n
Status: Downloaded newer image
n8n ready on 0.0.0.0, port 5678
```

Explanation: The named volume `n8n\_data` is mounted to `/home/node/.n8n` inside the container, which is where n8n stores its SQLite database, credentials, and encryption key — so removing or recreating the container does not delete the workflows.

Solved Task 5: Change the Default n8n Port

Problem:

Run n8n on port 8080 instead of the default 5678, because port 5678 is already in use.

Solution:

```
# npm installation (Linux/macOS)
export N8N_PORT=8080
n8n start

# Docker installation
docker run -it --rm --name n8n -p 8080:5678 \
-v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

Output:

```
n8n ready on 0.0.0.0, port 8080
Editor is now accessible via:
http://localhost:8080
```

Explanation: For an npm install, the `N8N\_PORT` environment variable tells n8n itself to listen on a different port. For Docker, n8n inside the container always listens on 5678; instead, the host-side port in `-p HOST:CONTAINER` is changed to 8080.

Solved Task 6: Check if n8n is Running on Localhost

Problem:

Verify from the terminal that n8n is up and responding, without opening a browser.

Solution:

```
curl -I http://localhost:5678
```

Output:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

Explanation: `curl -I` sends a HEAD request and prints only the response headers. A `200 OK` status confirms n8n's web server is running and reachable on port 5678.

Solved Task 7: Stop and Restart a Running n8n Docker Container

Problem:

Stop an n8n container that is running in the background, then start it again.

Solution:

```
docker ps
docker stop n8n
docker restart n8n
```

Output:

CONTAINER ID	IMAGE	STATUS	NAMES
3f1a9c2e7b41	docker.n8n.io/n8nio/n8n	Up 4 minutes	n8n

Explanation: `docker ps` lists running containers, `docker stop n8n` gracefully stops the container named n8n, and `docker restart n8n` stops and starts it again in one step, keeping the same container and its mounted volume.

Solved Task 8: Keep n8n Running in the Background with pm2

Problem:

Run an npm-installed n8n instance so it keeps running even after the terminal is closed.

Solution:

```
npm install pm2 -g
pm2 start n8n --name n8n-server
pm2 save
pm2 status
```

Output:

```
[PM2] Starting n8n in fork_mode (1 instance)
[PM2] Done.
█
█ id █ name █ mode █ pid █ status █
█
█ 0 █ n8n-server █ fork █ 8123 █ online █
█
```

Explanation: `pm2` is a process manager that keeps Node.js applications like n8n running in the background, automatically restarting them if they crash. `pm2 save` preserves the process list so it can be restored after a system reboot.

Practice Tasks (Solve on Your Own):

Task 1: List three real-life examples of tasks you personally do manually that could be automated with a workflow tool like n8n.

Task 2: In your own words, write a short comparison (5-6 lines) of traditional automation vs workflow automation, using an example from your own field of study.

Task 3: Install n8n on your machine using npm and take a screenshot of the terminal once it shows the "Editor is now accessible" message.

Task 4: Open the n8n Editor UI in your browser and identify (label with arrows/notes) the Canvas, Nodes Panel, Top Bar, and Left Sidebar.

Task 5: Install Docker on your machine (if not already installed) and confirm the installation using ``docker --version``.

Task 6: Run n8n using the ``docker run`` command shown in this manual, then verify the container is running using ``docker ps``.

Task 7: Create a ``docker-compose.yml`` file for n8n (as shown in this manual) and start it using ``docker compose up -d``.

Task 8: Change the default n8n port to 7000 using the ``N8N_PORT`` environment variable and confirm it in the browser.

Task 9: Stop your running n8n instance, then restart it, and record the exact commands you used for each step.

Task 10: Using ``curl -I``, check whether n8n is running on your machine and explain what HTTP status code you received.

Task 11: Explain, in 3-4 sentences, why data stored inside a Docker container is lost when the container is removed unless a volume is used.

Task 12: Install and configure ``pm2`` to keep your npm-installed n8n instance running, then confirm its status using ``pm2 status``.

Conclusion:

In this lab, we introduced Workflow Automation and compared it with traditional automation, then explored n8n as an open-source, self-hostable workflow automation tool — its key features, common use cases, and Editor UI. We then installed and set up n8n using three different methods: npm on Windows, npm on Ubuntu, and Docker, and learned how localhost and ports work in this context, including how to change n8n's default port. Finally, we practiced starting, stopping, and restarting n8n using npm, Docker, Docker Compose, and pm2, giving us a solid, practical foundation for building automated workflows in the upcoming labs.